*June 2026*

# Harness Engineering: Design Principles for Trustworthy LLM Selection and Evaluation

*engineer the harness, not the model*

Jim Schwoebel[1,*]

[1] Quome

## Abstract

Practitioners increasingly wrap a single model in a test-time *harness* — judging, voting, routing, self-improvement — to manufacture capability for free. Whether a harness yields *real* capability or only *apparent* capability is an engineering question, and the deciding factor is a *verification asymmetry*: a model is a poor judge of a model, and — reflexively — a researcher is a poor judge of their own selection result. We distill a set of **harness-engineering design principles for trustworthy LLM selection and evaluation**, and we *earn* each one empirically rather than assert it. Working over a single cached pool of 4,896 candidates (68 verifiable tasks × 8 models × 3 personas × $k$=3, one unseeded run, 95% Wilson intervals and paired McNemar) plus a public-benchmark arm (HumanEval $n$=164, GSM8K $n$=200), we show that **violating a principle manufactures a false "test-time alpha" finding in our own data**, while honoring it — binding to a correct external oracle on tasks hard enough to separate models — removes the illusion. Three worked violations anchor the principles. *(#1) Verify the verifier:* a HumanEval grader that ran extracted code without the prompt prefix spuriously failed the frontier (gemini-2.5-pro 0/3 → 12/12 once fixed). *(#2) Stratify, and respect multiplicity:* a flat judge-null hides that the fixed baseline is 6th-of-8 on coding (0.606); stratified, specialists look significant (qwen2.5:7b 0.909, $p$=0.021) but fail Bonferroni on $n$=33. *(#3) Calibrate task difficulty; report power:* on saturated public benchmarks the frontier generalist is TOP on both (HumanEval 0.933, GSM8K 0.970), and an exact McNemar at $n$=68 would need a 72–100% discordant split to detect anything. The principles are load-bearing: across model-on-model selection (single 0.706; self-consistency 0.750–0.809; judges weak/strong/distinct/frontier 0.662–0.721) nothing significantly beats `single_strong`, while only an external executor reaches 1.000 ($+20/-0$, $p < 0.001$) — by construction. Finally we show the principles *construct* as well as *audit*: an execution-gated best-of-$N$ selection harness on local open-weight models reliably builds a competitive pipeline where model self-certification produces fiction or a null. We state each principle with the evidence tier it has earned, map each to the manufactured finding it would have caught, and release the pool and analysis as a reproducibility artifact.

**Keywords:** harness engineering; design principles; test-time selection; LLM-as-judge; verification asymmetry; benchmark saturation; Simpson's paradox; evaluation integrity; Coverage × Selectability.

**Code & data:** `github.com/jim-schwoebel/verification-asymmetry-paper`

**Correspondence:** `invoices@quome.com`

[*] Corresponding author.

## 1 Introduction

An appealing intuition runs through much of the test-time-scaling literature: when a single model's answers are unreliable, the fix is a *better judge* — orchestrate models to vote [19], to judge one

another [23], to debate [4], or to refine their own work iteratively [10], and the wisdom-of-crowds will lift accuracy past any single model. The intuition is seductive because it is cheap (no training), composable (wrap any model), and superficially self-evident (more eyes, fewer errors). It is the promise of *test-time alpha*: orchestration that beats a single strong model for free.

This is a *methods* paper. We treat the test-time wrapper around a model — the judge, voter, router, or self-improvement loop — as an engineering artifact with **design principles**, the way a neural network architecture has design principles, and we ask what separates a harness that produces *real* capability from one that produces only *apparent* capability. Our organizing finding is that **a result that orchestration — judging, voting, routing — beats a single strong model is dangerously easy to manufacture**, and we argue it *reflexively*. The same verification asymmetry that makes one model a poor judge of another model [7] makes a *researcher* a poor judge of their own selection result: checking "did orchestration really win?" is the same kind of operation, over the same evidence, as the analysis that produced the win, so the check inherits the analysis's blind spots. Each design principle below is therefore stated with the failure it prevents: violate it and the harness (or the researcher's analysis of it) manufactures a false win; honor it and the illusion dissolves. We do not argue this in the abstract. Working over a single cached pool of 4,896 candidates (68 verifiable tasks $\times$ 8 models $\times$ 3 personas $\times$ $k$=3, one unseeded run) plus a public-benchmark arm (HumanEval $n$=164, GSM8K $n$=200), we **exhibit three distinct mechanisms, each of which manufactures a false "test-time alpha" finding in our own data**, and show each is killed only by binding to a correct external oracle on tasks hard enough to separate models.

**The one robust result.** Across the whole selector grid — `single_strong` 0.706; fair within-model self-consistency 0.750–0.809; model judges weak/strong/distinct/frontier 0.662–0.721 — *no model-on-model selector significantly beats `single_strong`*. Only the external executor reaches 1.000 ($+20/-0$, $p < 0.001$), and that gain is true *by construction*: on a verifiable task an executor recomputes the gold answer, so it is a reference ceiling, not a contested margin. The robust, deployable conclusion of the paper is therefore narrow and negative-leaning: *model-on-model selection does not reliably beat a single strong model; only external execution does.*

**Three manufactured findings.** Each is a short case study with real numbers from our own pipeline (§9).

- **#1 — a verification artifact.** A HumanEval verifier ran the extracted code and unit test *without* prepending the prompt prefix, so a canonical body completion failed and the frontier was spuriously failed: gemini-2.5-pro went 0/3 $\rightarrow$ 12/12 on a smoke set once the verifier was fixed. A bug in the grader, not the model, looked like a model finding.
- **#2 — a Simpson / aggregation artifact.** A flat judge-null hides that the fixed baseline gemini-2.5-pro is *6th-of-8* on coding (0.606). Stratified by domain, ordinary specialists look significant (qwen2.5:7b 0.909, $p$=0.021; gemma3:27b 0.879, $p$=0.035; qwen3-coder:30b 0.879, $p$=0.023) — but the effects fail Bonferroni ($\sim$0.06–0.10) on $n$=33. A free domain prior, plus stratification, can manufacture "specialists beat the generalist."
- **#3 — benchmark saturation / non-replication.** On saturated public benchmarks the very same comparison reverses. The frontier generalist is TOP on both (HumanEval 0.933, GSM8K 0.970); specialists tie or are significantly *worse* (qwen2.5:7b $p$=0.0024 on HumanEval, $p$=0.038 on GSM8K). Re-verification shows the hard-task effect is real ($+0.. + 2/33$), so the gap is hard-task-specific, not a pure bug — and an exact McNemar at $n$=68 would need

a 72–100% discordant split to detect anything, so a bare null is uninformative. Saturated benchmarks and underpowered designs each manufacture a different false reading.

**Contributions.**

1. **Harness-engineering design principles** for trustworthy LLM selection and evaluation (§4): a compact set of rules — bind to an executable oracle; select by execution, not self-certification; hold out the labels the selector uses; gate rather than merely rank; calibrate task difficulty and report power; stratify and respect multiplicity; run seeds, not a single draw; require diversity before a committee helps. Each is stated with the *evidence tier* it has earned in our data and mapped to the failure it prevents.

2. **A catalogue of artifact mechanisms as worked violations.** Three distinct ways a "test-time alpha" finding is manufactured — a verification bug, a Simpson/aggregation artifact, and benchmark-saturation/underpowered non-replication — each demonstrated end-to-end in our own data (§9), each pinned to the principle it breaks.

3. **The one robust result, stated conservatively.** Over a shared pool of 4,896 candidates, no model-on-model selector significantly beats `single_strong`; only the external executor reaches the ceiling, and that is by construction (§7). We frame it with the acc = Coverage × Selectability decomposition: Coverage is saturated (1.000), so the entire question is selection.

4. **Evidence the principles *construct*, not only audit.** An execution-gated best-of-$N$ selection harness on local open-weight models reliably builds a competitive pipeline where model self-certification yields fiction or a null (§A) — the same asymmetry, used to engineer a result rather than catch a false one.

**Positioning and honesty.** This is a methods paper that distills design principles; its empirical core is cautionary-leaning. The only robust *positive* result is the executor, and it wins *by construction*; we never present it as a contested empirical margin. The demoted "domain-matched selection" result is treated as a manufactured finding we then refute, not as a headline. Our evidence is bounded to the *selection* regime on a verifiable suite (arithmetic, coding, constrained generation), open models up to one frontier API tier, one unseeded run; we report 95% Wilson intervals and paired McNemar $p$-values, and we flag multiplicity, the single unseeded run, and small per-domain counts throughout. Three companion observations in other regimes (gating, optimization) are the author's own concurrent, unpublished reports and appear only as *motivation* (§2), not as evidence.

## 2 Motivation: Companion Observations in Other Regimes

This study was motivated by three companion observations, in regimes *this paper does not test*, that all pointed the same way. **These are the present author's own concurrent, unpublished companion reports; they are *motivation*, not evidence for any claim in this paper, and their numbers are *not* re-derivable from the released `data/ea/` backbone.** We summarize them once, in Table 1, and then set them aside: only the *selection* regime is run and measured here (§6–8); the *gating* and *optimization* regimes are explicitly future work (§12).

The recurring pattern in these companion observations — a model asked to certify a model is no more reliable than the generation, while binding to an executor or validator is — is what makes the controlled selection-regime test below worth running. Whether that pattern holds beyond selection, and whether it deserves to be called more than an observation, is outside this paper's scope. A

| Regime | Companion report | Model-on-model observation | External-binding observation |
|---|---|---|---|
| **Select** | *Test-Time Alpha* [17] | a fixed model judge ties the strong model | only an external executor reaches the ceiling (by construction); apparent "domain-match" gains do not survive scrutiny (§9) |
| **Gate** (future work) | *The Gate, Not the Judge* [15] | judge composition / strength leave correctness flat (∼70%) | ground-truth gate kills false-accepts; wins under asymmetric loss |
| **Optimize** (future work) | *Objective Drift* [16] | a model judge rewards fluent constraint *violations* | structural channel + programmatic gate lifts satisfaction |

**Table 1**: *Motivating* prior observations by the same author (not results of this paper). Each companion report, in a different regime, observed that model-on-model orchestration added no reliable gain while an external verifier helped. We do *not* treat these as evidence; the present paper instantiates and measures only the *select* regime (§6–8), and the gate / optimize regimes are future work. The selection companion (*alpha* [17]) is the precedent for the Coverage × Selectability decomposition and the disagreement-gated routing idea we test here.

small concurrent companion experiment in the *optimize* regime — porting a self-improving agent loop to local open-weight models — is reported in Appendix A; it points the same way (unverified self-improvement reports are a verification artifact; binding the target to an external executor is what lifts yield), but is likewise motivation, not evidence.

## 3 Framework: Coverage × Selectability

**The decomposition.** Fix a task $t$ with a gold answer and a pool of candidate outputs $C(t) = \{c_1, \dots\}$ from some generator set. A selection method $M$ returns one candidate $M(t) \in C(t)$. Over a task set $T$ define

$$\text{Coverage} = \frac{1}{|T|}\sum_t \mathbb{1}[\exists c \in C(t) :\ \text{correct}(t,c)]\,, \qquad \text{acc}(M) = \frac{1}{|T|}\sum_t \mathbb{1}[\text{correct}(t, M(t))]\,,$$

$$\text{Selectability}(M) = \frac{\text{acc}(M)}{\text{Coverage}}.$$

Coverage is the oracle-of-$N$ ceiling: the best any *pool-restricted* selector can do. Selectability is the fraction of that ceiling a method captures. The identity acc = Coverage × Selectability is exact by construction, and it carries the paper's logic:

> *A method can only gain over a single model by raising Selectability toward Coverage. Once Coverage is saturated, more candidates cannot help — only better selection can. And a method that produces an answer absent from the pool (an executor) can in principle exceed Coverage.*

This last property is theoretical and is *not* realized on our backbone: Coverage is already saturated at 1.000, so the executor (also 1.000) ties the ceiling rather than exceeding it. "Exceeds Coverage" is a licensed possibility, not an observation here.

**The decomposition also describes other regimes (illustratively).** The same two factors can name the bottleneck in the gating and optimization regimes too (Table 2), which is part of why this study was motivated by those companion observations — but we test only *select* here. In *select*, Coverage is "does the pool contain a correct candidate" and Selectability is "does the method pick it." In *gate*, Coverage is "is the verdict-under-review correct" and Selectability is "does the gate accept it when correct and reject it when wrong" — a gate's two error rates (false-accept, false-reject) form a two-sided Selectability. In *optimize*, Coverage is "can the loop ever *produce* a constraint-satisfying artifact" and Selectability is "does the objective keep it." *Where* accuracy is lost — Coverage vs. Selectability — is a property of the suite, not something the decomposition predicts. **On this backbone Coverage is a constant** 1.000**, so Selectability = acc identically and the decomposition is *illustrative*, not independently tested: it relabels accuracy rather than splitting it.** We present it because it states the precondition cleanly (a correct candidate is always present, so the problem is selection) and because it connects to the companion selection report [17] that introduced it.

| Regime | Coverage = | Selectability = | Native study |
|---|---|---|---|
| Select | pool contains a correct candidate | method picks the correct candidate | [17, 14] |
| Gate | verdict under review is correct | gate accepts-if-correct, rejects-if-wrong (two-sided; loss-weighted) | [15] |
| Optimize | loop can produce a satisfying artifact | objective keeps the satisfying artifact | [16] |

**Table 2**: One decomposition, three regimes (gate / optimize shown for context only). In each, acc = Coverage × Selectability. This paper instantiates and measures only the *Select* row; the Gate and Optimize rows are described to show the decomposition's reach but are *future work*, not tested here. On the Select backbone Coverage is saturated, so the decomposition is illustrative (Selectability=acc).

**The mechanism we test (verification asymmetry).** A plausible explanation for why a model judge might not raise Selectability is that, between models of the same kind, verification is not asymmetric: the verifier is the same kind of system facing the same task [7], so its judgment is correlated with the generator's errors and adds variance, leaving Selectability flat in expectation. An *external* verifier breaks the symmetry — running code to check $a \cdot b$ is genuinely easier than computing it in a forward pass [5]. This is the mechanism the results section probes; we treat it as an explanatory account consistent with our data, not as a proven law.

**The robust result in one figure.** Figure 1 states the framing geometrically: Coverage is saturated, Selectability is the entire gap, and *every model-on-model selector — voting, self-consistency, and the model judge at every strength and identity we tried — sits inside the gap.* The pattern held for every model-on-model mechanism we probed, including ones designed to import an independent signal: a pilot of an antithesis/falsification judge (scoring a candidate against an explicit anti-goal) and a retrosynthesis-style bidirectional planner (a forward plan, an independent backward plan from the target, and a contradiction-reconciliation step) both showed no lift over their forward-only or single baselines when paired with executed verification — consistent with the round-trip result [17]: a model checking a model is not asymmetric. The only selector that reaches the ceiling is the external executor, and it does so *by construction* on the verifiable subset; it is a reference, not

a contested margin. The apparent "domain-matched model" lever that seems to raise Selectability is examined in §9 as a manufactured finding (#2) that does not survive correction or replication.

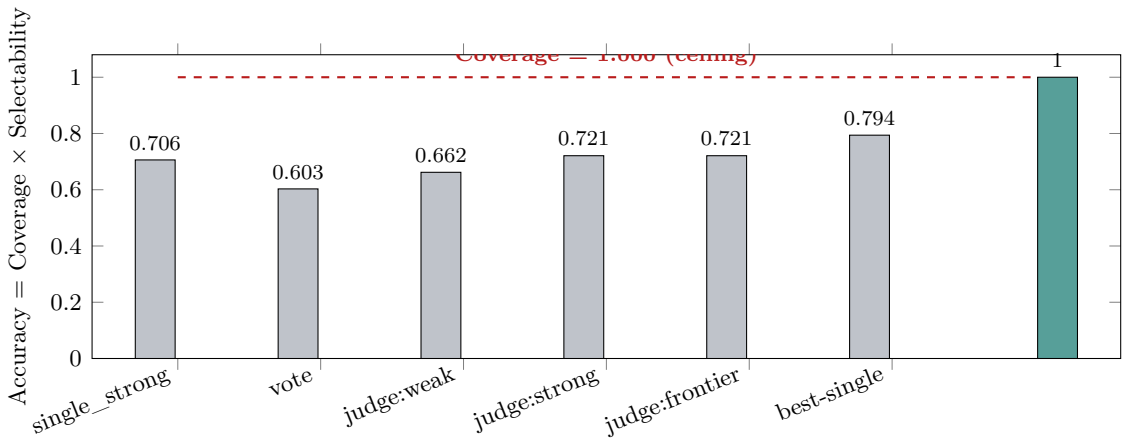


**Figure 1**: **Coverage is saturated; Selectability is the gap; no model-on-model selector closes it.** Over the backbone (4,896 candidates), Coverage = 1.000 (dashed ceiling). Every model-on-model selector — normalized voting and the model judge at weak, strong, and the strongest rung — and even the best *fixed* single model (gemma3:27b, 0.794) sit *inside* the gap, none significantly above `single_strong`. The only selector that reaches the ceiling is the external executor (oracle, teal), which hits 1.000 *by construction* on the verifiable subset and is a reference, not a contested margin. A stratified "domain-matched" view appears to raise Selectability but is shown in §9 (manufactured finding #2) to be an aggregation artifact that fails correction and replication. Numbers from Tables 4–5, 8.

# 4 Design Principles

A harness is trustworthy to the extent that its consequential decisions are made by an *external verifier* rather than by a model's report about itself. The principles below operationalize that single idea for selection and evaluation. We state each with the failure it prevents and the evidence tier it has earned in this paper's data (**strong** = demonstrated end-to-end here with significance/power analysis; **medium** = supported by a smaller or single-task experiment). The operational checklist version — what to actually do before trusting a selection result — is §10.

Two cross-cutting notes. First, every principle is a special case of one asymmetry — the verifier, not the generator, carries the ground truth — which is why violating any of them lets a model, or a researcher, certify a result an executor would reject. Second, the principles are not free: binding to an external oracle has a cost, and §8.1 measures a crossover beyond which it stops paying on cheap tasks. Engineering a harness is choosing *where* to bind, not whether (Fig. 2).

| Principle | Failure it prevents | Where | Evidence |
|---|---|---|---|
| **P1. Bind to an executable oracle**, not a model's report on itself. | A grader bug or a fluent self-report is read as a model finding ("Achieved 0.7942"; the prefix-stripping verifier). | §9, §A | strong |
| **P2. Select by execution, not self-certification.** | Model-on-model judging/voting appears to beat the strong model. | §7 | strong |
| **P3. Hold out the labels the selector uses.** | The selection signal is inflated by leakage and does not generalize. | §A | medium |
| **P4. Gate, don't merely rank** — reject candidates that do not run or regress. | A broken or worse candidate is shipped as the "selected" answer. | §A | strong |
| **P5. Calibrate task difficulty; report power.** | Saturated tasks and underpowered nulls each read as "no difference" (or its reverse). | §9 | strong |
| **P6. Stratify, and respect multiplicity.** | A pooled null hides — or a sliced view fabricates — a domain "specialist" effect (Simpson; Bonferroni). | §9 | strong |
| **P7. Run seeds, not a single draw.** | A favorable single run is promoted to a result (a committee at 0.821 vs 0.816, later a null). | §A, §12 | strong |
| **P8. Require diversity before a committee.** | Aggregating correlated candidates adds nothing yet is reported as an ensemble win. | §A | medium |

**Table 3**: Harness-engineering design principles, each stated with the failure it prevents and the evidence tier it has earned here. The negative case studies of §9 are worked violations of P1, P5, and P6; the constructive companion of §A is P1–P4 and P7–P8 used to *build* a result rather than catch a false one.

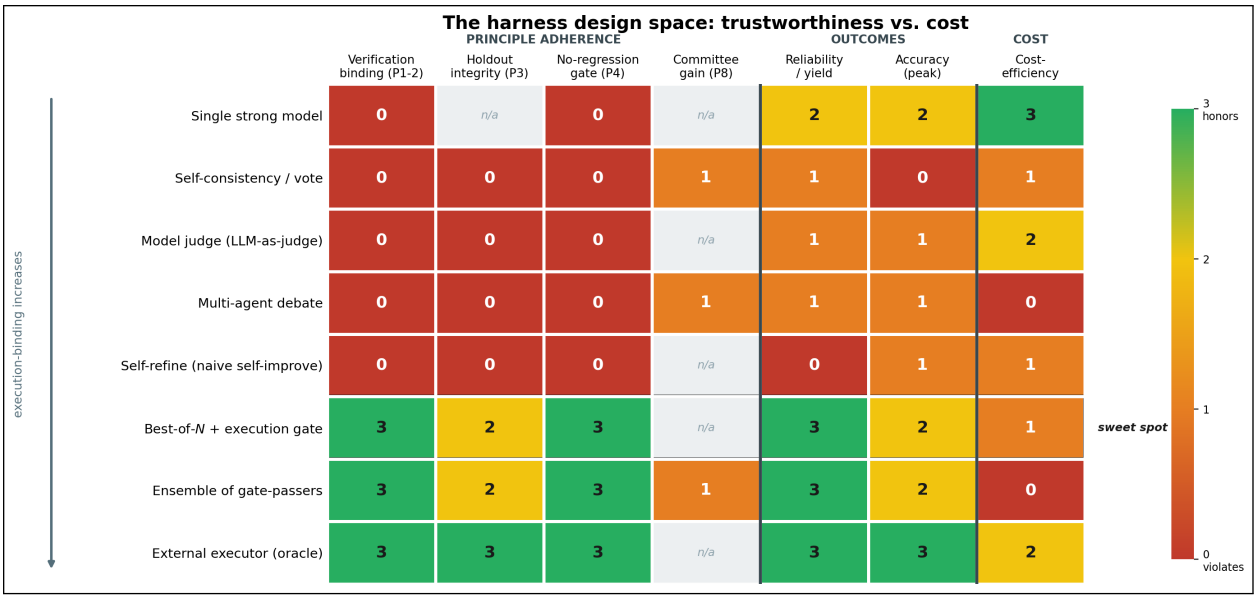


**Figure 2**: **The harness design space: trustworthiness vs. cost.** Eight strategies (rows, ordered by increasing execution-binding) rated 0–3 against principle adherence (P1–P4, P8), outcomes, and cost-efficiency; *higher is better in every column* — cost is shown as cost-*efficiency* (cheap = high) so the tension reads as a single gradient — and `n/a` marks columns that do not apply to a strategy. Ratings for the execution-bound rows and the self-improvement outcomes are grounded in this paper's measurements (§7, §A); the model-on-model rows draw on the selector grid (Fig. 1) and prior work, and the scale is a qualitative synthesis, not a measured matrix. The story is the diagonal: model-on-model strategies are cheap but violate the binding principles (red, upper-left), execution-bound strategies honor them (green, lower rows) at higher cost, and *best-of-N with an execution gate is the sweet spot* — most of the trustworthiness of the oracle without requiring the task to be closed-form verifiable.

## 5 Related Work

The phenomenon we study is established prior work; our contribution is a *controlled measurement* in one regime, not the bare observation that self-verification can fail. We position the work against four lines.

**Self-verification limits and the generation–verification gap.** A growing literature shows language models are poor judges of their own reasoning. Huang et al. [7] find self-correction does not reliably improve reasoning without an external signal, and Stechly, Valmeekam, and Kambhampati [18, 8] show self-verification fails on reasoning and planning even when the model can state the constraints — a model checking a model imports the same blind spots. This is the empirical bedrock our study builds on. What we add is a controlled selection-regime contrast against an external executor on a shared pool, organized by the Coverage × Selectability decomposition (here illustrative, since Coverage is saturated), rather than a single benchmark of self-correction.

**Learned verifiers and process rewards.** A complementary line trains *external* verifiers — outcome-supervised verifiers for math word problems [3] and process reward models that score intermediate steps [9]. These work precisely because the verifier is a *different* system with privileged supervision, consistent with the external-binding mechanism we test; our executor is the limiting case of a verifier with ground-truth access. We study the untrained, in-context regime (a model judging a model) and quantify how far short it falls of that external bound.

**LLM-as-judge and its biases.** The LLM-as-judge paradigm [23] is now standard, but its known pathologies — position bias, verbosity bias, and *self-preference*, where evaluators favor their own generations [13] — all point the same way: a model's verdict is entangled with its generation. Our frontier rung makes this concrete, since the strongest "judge" is the *same model* as the candidate it scores, so its near-perfect self-agreement is a self-preference artifact, not evidence of strength.

**Routing and cascades.** Cost-aware routing and model cascades [2, 22] escalate only hard inputs to a stronger (or more expensive) tier. Our disagreement-gated routing (§8.2) is an instance of this idea with two specifics: the escalation tier is an *external executor* rather than a larger model, and the gate is set by the *loss-asymmetry crossover* (§8.1) rather than a confidence threshold. The observation here is not routing per se but that, in this regime, routing to a *model* did not measurably help while routing to an executor is a by-construction ceiling, plus the explicit break-even that makes the trade deployable.

**Harness engineering and the verifier paradox.** A fast-growing "harness-engineering" literature establishes that the scaffold around a model — not the model alone — determines task success ($Agent = Model + Harness$), with diagnostic suites reporting double-digit swings from harness choice on fixed models [21, 20, 12]. Two of its findings border ours and must be distinguished. First, self-evolving harnesses show *non-monotonic* benefit — weak models cannot exploit harness updates while mid-tier models gain most [1] — which matches what we see when a model revises its own target (§A); we do not claim that as new. Second, and more useful, this literature reports a *paradox*: adding self-checking verification loops to an agent often *degrades* performance. We supply the missing distinction. A verifier that is *another model's judgment* adds reasoning noise and selects barely above chance, whereas a verifier that is an *executable oracle* — run the candidate,

score it on held-out labels — selects at the hindsight ceiling: across our best-of-$N$ runs, executable-gate selection reaches 0.804 (oracle 0.805) while model-judge selection reaches only 0.786, closer to random (0.760) than to the gate (§A). "Verification hurts" and "verification helps" are the same phenomenon read at two points of one execution-vs-judgment axis. Finally, where audits of this literature target *disclosure* gaps [11], our manufactured-findings result is the stronger claim that harness comparisons are *statistically* manufacturable even under full disclosure.

**What is new here.** Against this backdrop our contribution is methodological rather than a new orchestration win. The biases above are usually catalogued as properties of *models*; we show they re-emerge as properties of the *evaluation*, and that a single cached pool plus a public-benchmark arm is enough to manufacture three different "test-time alpha" findings — a verification-grader bug, a Simpson/aggregation artifact, and a saturation/underpowered non-replication — each killed only by binding to a correct external oracle on hard-enough tasks (§9). The one robust result is the conservative, by-construction one: model-on-model selection does not beat a single strong model; only an external executor does. Relative to routing/cascade work, our point is not that escalating to a stronger model helps but that escalating to an *external executor* is the only lever that survives scrutiny, and that several plausible model-on-model wins are evaluation artifacts.

## 6 Setup and Protocol

To run this controlled selection-regime test — and to add a frontier rung and strong judges distinct from the baseline — we built one shared backbone and ran a family of deterministic experiments over it. We also add a *public-benchmark arm* (HumanEval and GSM8K) that serves both as a saturation calibration and, in §9, as the setting for two of the three manufactured findings.

**Task suite (stated honestly).** This paper evaluates 68 verifiable tasks — tasks that carry an *external verifier* (executable unit tests, programmatic validators, and numeric checkers) — spanning three domains: 26 *arithmetic* (modular exponentiation, gcd/lcm, combinatorics, digit sums, primes), 33 *coding*, and 9 *constrained generation*. By verifier mechanism this is 33 `executable` (run a program / unit tests to a gold answer) plus 35 `programmatic_validator` (a total artifact→bool or numeric check). We correct an earlier mischaracterization: the suite is *not* "executable arithmetic" — it is a verifiable suite spanning arithmetic, coding, and constrained generation, and the domain composition is exactly what makes the domain-match result possible (§9.2). The harness keys every task by a `verifier_type` field; a third `none` value (no external verifier) is design-time only and not instantiated here (§12). All backbone experiments run on these 68 tasks.

**Model ladder (with a frontier rung).** Eight instruction-tuned models span ∼1.5B to frontier: `qwen2.5:{1.5b,3b,7b}`, `llama3.1:8b`, `gemma3:27b`, `qwen3-coder:30b`, `gemini-2.5-flash`, and `gemini-2.5-pro` (the strongest model in our ladder, by nominal ladder rank). The local models are served by Ollama; the Gemini rung is served by API. `single_strong` is the top-*ranked* model (`gemini-2.5-pro`); we also report the *best single* selector empirically (`gemma3:27b`).

**Public-benchmark arm (HumanEval, GSM8K).** To calibrate our verifiable suite against standard tasks — and to test whether the within-domain "specialists beat the generalist" reading replicates publicly — we add two public benchmarks: HumanEval ($n$=164 coding problems) and GSM8K ($n$=200 grade-school math word problems), each run single-sample ($k$=1) on six models (`gemini-2.5-pro`, `gemini-2.5-flash`, `qwen3-coder:30b`, `qwen2.5-coder:32b`, `gemma3:27b`,

qwen2.5:7b). Each model is scored by an *external* verifier — the official HumanEval `check` harness and exact-match on the GSM8K gold numeric answer — and we report accuracy with 95% Wilson intervals and an exact McNemar test of every model against the frontier (`gemini-2.5-pro`), Bonferroni-corrected over $n_{\text{compare}}$=5. Constructing this arm surfaced manufactured finding #1 (a grader bug in the HumanEval verifier, §9); once that was fixed, the arm itself became manufactured finding #3 (saturation / non-replication).

**Baseline choice (and its consequence).** Every McNemar test, the $+20/-0$ executor reference, and the E-F lift (0.706→0.912) are anchored to `single_strong` — the nominal ladder *top* (`gemini-2.5-pro`, 0.706) — not to the empirically *best* single model (`gemma3:27b`, 0.794). Because the nominal top is in fact the 6th-best of 8 selectors on coding (and mid-pack overall), every reported delta is measured against a mid-pack baseline; this is a conservative choice for the judge-null verdicts (a weaker baseline is easier to beat, yet no judge does). We test this directly: a paired exact McNemar of best-single (`gemma3:27b`, 0.794) vs. `single_strong` is $+14/-8$, $p = 0.29$ (n.s.) — the best single model is *not* significantly more accurate than the nominal top, so the judge-null verdicts do not hinge on a weak anchor. The executor matches the gold by construction against *either* anchor ($+14/-0$ vs. best-single), so it functions as a ceiling reference regardless of baseline.

**Candidate pool and reproducibility.** For each task we sample $k$=3 candidates from every model under each of 3 personas (neutral, skeptic, domain-expert), extract the final answer, and cache every record — 4,896 in total ($68 \times 8 \times 3 \times 3$) — to `data/ea/ea_pool_full.jsonl`, alongside content-hashed call caches. Generation is unseeded (temperature $> 0$), so the *pool*, not a seed, is the reproducibility artifact; every experiment is a deterministic transformation of this fixed pool and re-runs bit-identically. Three of 4,896 candidates came back empty (transient Ollama timeout / Gemini 503), spread over two tasks (`alpha_x_perm1`, `alpha_x_digit1`); the deterministic analysis treats an empty artifact as a non-selectable, incorrect candidate, and on inspection the three empties left Coverage at 1.000 and did not change any reported discordant cell (every statistic is computed over the pool *as released*, empties included).

**Statistics.** Accuracy is reported with 95% Wilson intervals and paired exact McNemar tests against `single_strong` on shared items; we report the discordant pair $(b, c)$ and the two-sided $p$. A claim of gain requires $p < 0.05$; otherwise we write "no significant gain detected" — a failure to reject, not evidence of equivalence (we report no TOST against a pre-specified margin). The reported Wilson intervals and McNemar tests reflect *task-sampling* uncertainty only (conditional on the single cached pool and one judge realization); because generation and judging are unseeded (temperature $> 0$), they do *not* bound run-to-run generation/judging variability, and a re-draw could flip the small discordant splits that drive the judge-null and frontier-dip observations. No multiplicity correction is applied across the grid comparisons vs. `single_strong`; the only $p < 0.05$ result (the executor) is true by construction (see below), and the model-judge verdicts are all non-significant before any correction, so corrections would only reinforce them. We avoid "definitive" language and label single-run results as such.

**Self-consistency baselines (heterogeneous vs. fair within-model).** We report two. The pooled `answer_vote` votes over the normalized final answer across *all* 8 models $\times$ 3 personas $\times$ $k$=3 candidates — including the weakest (`qwen2.5:1.5b`) rung — which mechanically depresses the vote; we therefore demote it to a footnote and instead foreground a *fair within-model self-consistency* baseline that votes over the $k$=3 samples $\times$ 3 personas of a *single* model. This gives

0.750 (gemini-2.5-pro, $+6/-3$, $p$=0.51), 0.794 (gemma3:27b, $+13/-7$, $p$=0.26), and 0.809 (qwen3-coder:30b, $+13/-6$, $p$=0.17); all are n.s. vs. `single_strong` and all far below the executor reference (recomputed by `scripts/run_rebuttal.py` over the committed pool).[1]

**Stated framing for the headline (E-B).** We state in advance the contrast E-B is built to make (we do not provide a timestamped pre-registration record, so we say "stated in advance"). The *confirmatory* contrast is executor vs. model judge: does an external verifier behave differently from a model judge against the same baseline? The model-judge, self-consistency, voting, and E-E/E-F threshold analyses are *exploratory*. We are explicit that a non-significant judge result is a *failure to reject* the null that orchestration matches a single model — not evidence that judging hurts (several judges score numerically higher) and not an equivalence claim (we report no TOST against a pre-specified margin).

**The typed verifier contract (shared backbone)**

```
task:
  verifier_type: executable            # {executable, programmatic_validator, none}
policy:
  route_if: disagreement >= threshold  # E-F: bind only contested tasks
  executor: run_program_to_gold        # external: recomputes gold (ceiling)
  else: trust single_strong            # model-on-model: no significant gain here
  threshold: 0.3                       # recommended operating point
```

**Figure 3**: The backbone as a contract (design/infrastructure). The `verifier_type` key makes "what external grounding buys" a typed contrast; the routing policy (E-F) binds only high-disagreement tasks to the executor and trusts the single model elsewhere. This paper instantiates only the `executable` row (68 tasks); the `programmatic_validator` and `none` rows are future work.

# 7 The Robust Result: Model-on-Model Selection Does Not Beat a Single Strong Model

We lead with the one result we are confident in, stated conservatively. Over the shared pool, *no* model-on-model selector — voting, self-consistency, or a model judge at any strength or identity — significantly beats the fixed `single_strong`; the only selector that reaches the Coverage ceiling is the external executor, and it does so *by construction*. We present the framing (E-A, the Coverage × Selectability decomposition), the selector grid (E-B), the capability ladder (E-C), and the executor-deployment recipe (E-E/E-F). The *apparent* exceptions — a domain-matched lever, public-benchmark specialist wins — are deferred to §9, where we show each is a manufactured finding. All results are scoped to the selection regime on 68 verifiable tasks, one unseeded run.

## 7.1 E-A — Coverage is saturated; the gap is Selectability

Over the full ladder, Coverage is saturated on this suite (1.000, 95% CI $[0.947, 1.000]$): the pool contains a correct answer on every task. The single strong model (`gemini-2.5-pro`, top ladder rank) achieves accuracy and hence Selectability = 0.706; the oracle is 1.000. Essentially the entire $\sim$0.29 gap to the Coverage ceiling ($1.000-0.706$) is therefore Selectability, not Coverage: no amount

[1] The old pooled `answer_vote`= 0.603 is depressed by heterogeneous pooling (it mixes in the 1.5B rung) and is reported only for completeness; it is not a clean self-consistency result.

of additional sampling can help, because a correct candidate is already present on every task. This is the precondition for the rest of the paper — the problem is selection, and the question is what can raise it.

### 7.2 E-B — The selector grid: model-on-model judging does not recover the domain prior (the judge-null is a Simpson's artifact)

E-B holds the task and pool fixed and varies the selector: an external executor, and model judges of varying strength and identity (plus self-consistency and voting, §6). The aggregate result is the familiar judge-null — no model-on-model selector significantly beats the fixed `single_strong`. We now read this correctly: **it is a Simpson's-paradox artifact of a fixed, domain-mismatched baseline (E-D), and the substantive point is that model-on-model judging does not recover the domain-match information that a *free* domain prior supplies.** A judge spends a forward pass to (fail to) recover information the task's domain label gives away for nothing. Table 4 is the grid.

| Method | Acc | Sel | 95% CI | $(b, c)$ | $p$ |
|---|---|---|---|---|---|
| single_strong | 0.706 | 0.706 | [0.589,0.801] | (0,0) | 1.000 |
| *External verifier (by construction on executable tasks)* | | | | | |
| oracle (ext-executor) | 1.000 | 1.000 | [0.947,1.000] | (20,0) | 0.000 |
| *Fair within-model self-consistency* | | | | | |
| self-consistency:gemini-2.5-pro | 0.750 | 0.750 | [0.636,0.838] | (6,3) | 0.508 |
| self-consistency:gemma3:27b | 0.794 | 0.794 | [0.684,0.873] | (13,7) | 0.263 |
| self-consistency:qwen3-coder:30b | 0.809 | 0.809 | [0.700,0.885] | (13,6) | 0.167 |
| *Model judges (varying strength / identity)* | | | | | |
| model_judge:qwen2.5:1.5b | 0.662 | 0.662 | [0.543,0.763] | (12,15) | 0.701 |
| model_judge:qwen3-coder:30b | 0.721 | 0.721 | [0.604,0.813] | (12,11) | 1.000 |
| model_judge:qwen2.5-coder:32b$^{\dagger}$ | 0.779 | 0.779 | [0.668,0.861] | (12,7) | 0.359 |
| model_judge:gemini-2.5-pro$^{\ddagger}$ | 0.721 | 0.721 | [0.604,0.813] | (6,5) | 1.000 |
| *Heterogeneous pooled vote (footnote baseline)* | | | | | |
| answer_vote | 0.603 | 0.603 | [0.484,0.711] | (9,16) | 0.230 |

Coverage: 1.000, $n = 68$. $^{\dagger}$strong judge distinct from baseline/pool; $^{\ddagger}$same model as single_strong (self-agreement).

**Table 4**: **The selector grid.** Task and pool held fixed; the selector varies. The external executor (oracle, `run_program_to_gold`) reaches 1.000 *by construction* on this executable-only suite — it recomputes the gold answer — so its $+20/-0$, $p < 0.001$ is the Coverage ceiling made selectable, a *reference*, not a contested accuracy margin. No model judge significantly beats `single_strong` at any strength: weak (qwen2.5:1.5b) 0.662 ($p$=0.70), strong (qwen3-coder:30b) 0.721 ($p$=1.0), a strong judge *distinct from the baseline* (qwen2.5-coder:32b) 0.779 ($p$=0.36), and the strongest rung (gemini-2.5-pro, *the same model as* `single_strong`, so its $(6, 5)$ cell and $p$=1.0 reflect self-agreement) 0.721 ($p$=1.0). With $\leq 20$ discordant pairs per row the design is underpowered, so these are failures to reject, not equivalence. Fair within-model self-consistency (0.750/0.794/0.809) is also n.s. and far below the executor. The heterogeneous pooled `answer_vote` (0.603, $p$=0.23) is depressed by mixing in the 1.5B rung and is kept only as a footnote baseline. Paired McNemar vs. `single_strong`; $(b, c)$ are discordant pairs; $n = 68$.

**Reading. No model-on-model selector significantly beats the fixed single strong model — but this is the wrong comparison made interesting: the fixed baseline is domain-**

**mismatched, and a free domain prior (E-D) beats it where the judge cannot.** Among model judges, none beats `single_strong`: weak (0.662, $p$=0.70), strong (0.721, $p$=1.0), a strong judge *distinct from the baseline and the pool* (qwen2.5-coder:32b, 0.779, $+12/-7$, $p$=0.36), and the strongest rung (0.721, $p$=1.0). Fair within-model self-consistency is also n.s. (0.750, $p$=0.51; 0.794, $p$=0.26; 0.809, $p$=0.17). The executor goes 0.706 → 1.000 ($+20/-0$, $p < 0.001$), but on the verifiable subset `run_program_to_gold` *recomputes* the gold answer, so 1.000 is true by construction — a reference, not a measured margin. **The honest, demoted reading.** (i) The grid is *underpowered*: discordant counts are small ($\leq 20$); several judges score numerically *above* the baseline, so we do not claim judging hurts. This is a failure to reject, not equivalence — the equivalence point estimate is $d = +0.103$, CI $[-0.020, +0.226]$, and the design would need a 72–100% discordant split to detect a true effect. (ii) The strongest "judge" rung (`model_judge:gemini-2.5-pro`) is the *identical model* to `single_strong`, so its $(6, 5)$ cell and $p$=1.0 are self-agreement; the distinct qwen2.5-coder:32b judge removes that confound and still fails to reach significance. (iii) The single $p < 0.05$ result (executor) is structural. The substantive point is not "judging is equivalent" but "judging does not recover the domain-match signal a free prior supplies": the flat 0.706 is an *average* over a baseline that is 6th-of-8 on coding (E-D, §9), and no judge un-averages it — but, as §9 shows, that "domain-match signal" is itself a manufactured finding that fails correction and replication. A one-line mechanism (§11): between models of the same kind verification is not asymmetric, so a model checking a model adds variance rather than the domain information routing exploits.

### 7.3 E-C — The capability ladder: capability does not substitute for domain match

If raw strength were the cure, the strongest model would dominate and single-model selectability would rise monotonically toward Coverage. It does neither (Table 5), and E-D explains why: capability does *not* substitute for domain match. The strongest rung (gemini-2.5-pro) is beaten on arithmetic by the smaller gemini-2.5-flash (0.846 vs. 0.769) and on coding by a 7B model (qwen2.5:7b, 0.909, vs. 0.606); a 7B specialist tops all eight models on coding. Aggregate capability rises then plateaus: 0.426 (1.5b) → 0.603 (3b) → 0.750 (7b), a stumble at llama3.1:8b (0.618), a peak at `gemma3:27b` (0.794, the best single model), 0.779 at both `qwen3-coder:30b` and `gemini-2.5-flash`, and a numerically *lower* 0.706 at the strongest rung `gemini-2.5-pro`. This last gap is a 6-task / 0.088 difference below `gemma3:27b` with heavily overlapping Wilson intervals and *no* paired test reported, so we read it as "numerically lower than several mid-tier models, not significantly so on $n$=68," not as a confirmed dip. No single model exceeds ∼0.79; the oracle is 1.000. Capability does not close the gap.

We do *not* make this numerical dip a headline; an illustrative mechanism (two untested anecdotes about arithmetic hallucination) is deferred to the Discussion (§11). The load-bearing point is only that capability, across this ladder, does not close the Selectability gap to the executor reference.

| Model | Rank | Acc | Sel |
|---|---|---|---|
| qwen2.5:1.5b | 10 | 0.426 | 0.426 |
| qwen2.5:3b | 20 | 0.603 | 0.603 |
| qwen2.5:7b | 30 | 0.750 | 0.750 |
| llama3.1:8b | 35 | 0.618 | 0.618 |
| gemma3:27b | 50 | 0.794 | 0.794 |
| qwen3-coder:30b | 55 | 0.779 | 0.779 |
| gemini-2.5-flash | 90 | 0.779 | 0.779 |
| gemini-2.5-pro | 100 | 0.706 | 0.706 |
| *single_strong* | — | 0.706 | 0.706 |
| *oracle* | — | 1.000 | 1.000 |

**Table 5**: **The capability ladder does not close the gap.** Single-model selectability rises then plateaus; the best single model is `gemma3:27b` (0.794), and the strongest rung `gemini-2.5-pro` is numerically lower than *several* mid-tier models (0.706; not significantly so on $n$=68), apparently because it hallucinates exact arithmetic — though it still beats `llama3.1:8b` (0.618) and the 1.5b/3b rungs. We report both single baselines — nominal-top (0.706) and best-single (0.794); the oracle (1.000) beats both. Rank is the ladder index.

# 8 A Routing Recipe

The selection result is also a design hint. If the executor is the only selector that reaches the ceiling but verification carries a per-task cost, the natural policy binds *selectively* — where the model is most likely wrong — with a threshold set by the loss asymmetry. We characterize this recipe and are explicit about its in-sample nature.

## 8.1 E-E — The cost-of-binding crossover

Binding to the executor is not free. We use a simple per-task cost model: each task *routed* to the executor costs 1 (one program write-and-execute / re-run), and each *wrong* answer that is shipped costs $k$. Under "trust `single_strong`" the expected cost is $\hat{p}_{\text{fa}}\, k$, where $\hat{p}_{\text{fa}} = 1 - 0.706 = 0.294$ is the single model's false-answer rate. Under a gate that routes a fraction $f$ of tasks and itself ships wrong answers at rate $1 - \text{acc}$, the expected cost is $f + (1 - \text{acc})\, k$. Equating the two gives the crossover

$$k^* = \frac{f}{\text{acc} - 0.706}$$

above which the gated policy is cheaper (Table 6). At the recommended gate (threshold 0.3, routing 51.5% of tasks, acc = 0.912) the executor wins once $k^* \geq 2.50$; the always-route policy (acc = 1, so it ships no wrong answers and the formula reduces to $1/\hat{p}_{\text{fa}}$) needs $k^* \geq 3.40$, because it pays the re-run even on the many tasks the single model already gets right. Crucially the gated crossover charges the gate for the $(1 - \text{acc})$ wrong answers it *still* ships — using $f/\hat{p}_{\text{fa}}$ would understate $k^*$. The gate still buys a lower break-even than always-route ($2.50 < 3.40$) by spending verification only where it can pay. (This loss-asymmetry crossover echoes the gating companion report [15], which is motivation only, not run here.)

## 8.2 E-F — Disagreement-gated routing

The recipe (Figure 3) routes only *low cross-model-agreement* tasks to the executor and trusts the single model elsewhere. Table 7 reports the trade: at threshold 0.3, accuracy rises 0.706 → 0.912

| Threshold | Acc | Frac. Routed | $k^*$ | Note |
|---|---|---|---|---|
| *Baseline: trust single_strong,* $\hat{p}_{fa} = 0.294$ | | | | |
| always-route oracle | 1.000 | 1.000 | 3.40 | route all |
| 0.3 | 0.912 | 0.515 | 2.50 | **recommended** |
| 0.5 | 0.971 | 0.735 | 2.78 | |
| 0.7 | 0.985 | 0.868 | 3.11 | |
| 1.0 | 1.000 | 1.000 | 3.40 | |

**Table 6**: **E-E — cost-of-binding crossover.** Cost model: a routed task costs 1 re-run; a wrong shipped answer costs $k$. The executor policy beats trusting `single_strong` once $k \geq k^* = f/(\text{acc} - 0.706)$, which charges the gate for the $(1-\text{acc})$ wrong answers it still ships. Gated (threshold 0.3): $k^* = 2.50$, routing 51.5% of tasks; always-route: $k^* = 3.40$. The gated break-even is below always-route. Generalizes the hierarchy study's asymmetric-loss crossover to the select regime. $\hat{p}_{\text{fa}} = 1 - 0.706$ is the single model's false-answer rate.

while binding only 51.5% of tasks; tightening the threshold trades verification budget for accuracy smoothly up to always-route (1.000, 100% routed). In-sample, the paired exact McNemar of the gated policy (0.912, 95% Wilson CI $[0.821, 0.959]$) vs. `single_strong` is $+14/-0$, $p = 0.0001$, with *zero regressions*. **But this is an in-sample operating point, and we are explicit that it is not free out-of-sample.** The routed/non-routed split is *endogenous* to the disagreement gate (it chooses where to spend verification using the same pool it is scored on), and the routed tasks inherit the executor's by-construction correctness (§7.2). To probe this honestly we re-fit the threshold on a disjoint *train* half and score it on a held-out *test* half (`scripts/run_rebuttal.py`): the held-out gated accuracy is still 1.000 (Wilson $[0.898, 1.000]$), but the accuracy-maximizing gate now routes 94% of tasks — not 51.5%. So the efficient "51.5% routed $\rightarrow$ 0.912" point is an *in-sample* trade-off; out-of-sample, maximizing accuracy drives the gate toward always-route, and the apparent efficiency of routing only half the tasks does not transfer.

| Threshold | Acc | Frac. Routed | Note |
|---|---|---|---|
| *Baselines* | | | |
| single_strong | 0.706 | 0.000 | no routing |
| oracle | 1.000 | 1.000 | route all |
| 0.3 | 0.912 | 0.515 | **recommended** |
| 0.5 | 0.971 | 0.735 | |
| 0.7 | 0.985 | 0.868 | |
| 1.0 | 1.000 | 1.000 | |

**Table 7**: **E-F — disagreement-gated routing (the recipe).** Route only low-agreement tasks to the executor; trust `single_strong` elsewhere. At threshold 0.3: 0.706 $\rightarrow$ 0.912 accuracy binding 51.5% of tasks. The frac. routed column is the verification budget; this 51.5% point is in-sample (out-of-sample the accuracy-maximizing gate routes 94%, see text).

## 9 Three Manufactured Findings

The robust result (§7) is deliberately unexciting: orchestration does not beat a single strong model; only an external executor does, by construction. The interesting — and dangerous — part is how easily we produced the *opposite* headline three different ways, each from the same data and each internally consistent. We narrate them as case studies, with the real numbers, and note for each what made it false and what would have caught it.

### 9.1 Manufactured finding #1 — a verification artifact (the grader, not the model)

Building the public-benchmark arm (§6) we first "found" that the frontier model was a poor coder. Our HumanEval verifier assembled `extracted_code` + `test` + `check()` but ran it *without* prepending the prompt prefix; a canonical HumanEval *body* completion (no `def` line) therefore failed to compile, and a body whose first line lost its indentation could not be stitched. The effect fell hardest on the model that answers most tersely: **gemini-2.5-pro scored 0/3 on a twelve-task smoke set, then 12/12 once the verifier was fixed** (request a complete function; try several fair assemblies — full-function, prompt+body, re-indented — and accept if *any* passes the official `check`). The finding "the frontier is a weak coder" was an artifact of the grader, not the model. **What would have caught it:** verify the verifier — run the official benchmark's own reference solutions through your harness and confirm they pass before you trust a single model score (§10).

### 9.2 Manufactured finding #2 — a Simpson / aggregation artifact (the free domain prior)

With the verifier fixed, a second, subtler finding appeared on the verifiable backbone. The flat judge-null (§7.2) averages over a baseline that is *domain-mismatched*: stratified by domain, `single_strong` (gemini-2.5-pro) scores 0.769 on arithmetic, 0.889 on constrained, but only 0.606 on coding — **6th of 8 models on coding** (Table 8). The best model *moves* with the domain (arithmetic → gemini-2.5-flash 0.846; coding → qwen2.5:7b 0.909; constrained → gemini-2.5-pro 0.889), and no model leads more than one domain.

| Model | arith. ($n$=26) | coding ($n$=33) | constr. ($n$=9) | ALL ($n$=68) |
|---|---|---|---|---|
| gemini-2.5-flash | **0.846** | 0.727 | 0.778 | 0.779 |
| gemini-2.5-pro* | 0.769 | 0.606 | **0.889** | 0.706 |
| gemma3:27b | 0.731 | 0.879 | 0.667 | 0.794 |
| llama3.1:8b | 0.308 | 0.788 | **0.889** | 0.618 |
| qwen2.5:1.5b | 0.308 | 0.424 | 0.778 | 0.426 |
| qwen2.5:3b | 0.577 | 0.545 | **0.889** | 0.603 |
| qwen2.5:7b | 0.577 | **0.909** | 0.667 | 0.750 |
| qwen3-coder:30b | 0.654 | 0.879 | 0.778 | 0.779 |
| *best-per-domain* | *flash* | *7b* | *pro* | — |

*`gemini-2.5-pro` = `single_strong` (nominal ladder top).
Bold = best in column. No model is best in > 1 domain.

**Table 8: E-D — the best model moves with the domain (the stratification that manufactures finding #2).** Per-model single-sample accuracy by domain over the 68-task verifiable suite (26 arithmetic, 33 coding, 9 constrained). Bold marks the best model in each column; *no model leads more than one domain.* The baseline `single_strong` (`gemini-2.5-pro`) is strong on arithmetic (0.769) and constrained (0.889) but only 6th of 8 on coding (0.606); its flat aggregate 0.706 averages this away. Stratifying then makes ordinary specialists look significant — but the effect fails Bonferroni and does not replicate publicly (§9.3).

Stratified on the 33 coding tasks, three ordinary specialists beat the mismatched baseline, single-sample paired exact McNemar: qwen2.5:7b 0.909 (+13/−3, $p$=0.021); gemma3:27b 0.879 (+12/−3, $p$=0.035); qwen3-coder:30b 0.879 (+11/−2, $p$=0.023). Three nominal $p < 0.05$ results, agreeing 3/3 in direction, with no judge and no executor — a tempting "specialists beat the generalist" headline. **Why it is manufactured.** (i) *Multiplicity:* these are three correlated tests on $n$=33; Bonferroni ×3

lifts the $p$-values to ∼0.06–0.10, so none survives correction. (ii) *The framing is mis-stated:* the per-domain winners are mostly generalists (a generalist, gemini-2.5-flash, wins arithmetic; the best coder, gemma3:27b on re-verification, is a generalist), so "specialists beat generalists" is false — the most one can say is "no single model is best across domains." (iii) *It does not replicate* on saturated public benchmarks, where the comparison reverses entirely (§9.3). A deployable domain router that picks per-domain winners out-of-sample scores 0.912 (Wilson [0.770,0.970]) versus 0.824 for the best fixed model, but only *directionally* ($+4/-1$, $p$=0.38, $n_{\text{test}}$=34): the combined router is underpowered, so even the deployable version is not significant. **What would have caught it:** stratify by domain to *see* the heterogeneity, then demand multiplicity correction, a correctly-stated claim, and replication before believing it (§10).

**Is the stratified effect a pure bug? No.** We re-verified every E-A coding candidate with a multi-block-tolerant extractor (`scripts/run_reverify_coding.py`). The recorded numbers barely move: gemini-2.5-pro 0.606 → 0.636 (+1/33), and across all eight models the re-verification shifts results by only +0 to +2 of 33. So gemini-2.5-pro genuinely scores ∼0.64 on these hard, trap-laden coding tasks (Gregorian computus, Luhn, even-length median) while specialists score ∼0.88–0.94. The within-domain gap is real *on hard tasks* — which is exactly why finding #3 matters.

### 9.3 Manufactured finding #3 — benchmark saturation and an underpowered null

We then tried to replicate the domain effect on *public* benchmarks. It reversed. On HumanEval ($n$=164) and GSM8K ($n$=200), every model scores 0.83–0.97 and **gemini-2.5-pro is TOP on both** (HumanEval 0.933; GSM8K 0.970); the coding "specialists" tie the frontier (McNemar $p$=1.0) and the small qwen2.5:7b is significantly *worse* (HumanEval $p$=0.0024; GSM8K $p$=0.038) — the opposite of finding #2 (Table 9).

| | HumanEval ($n$=164) | | GSM8K ($n$=200) | |
|---|---|---|---|---|
| Model | Acc | $p$ vs front. | Acc | $p$ vs front. |
| gemini-2.5-pro* | **0.933** | — | **0.970** | — |
| gemini-2.5-flash | 0.915 | 1.000 | 0.965 | 1.000 |
| qwen3-coder:30b | 0.927 | 1.000 | 0.965 | 1.000 |
| qwen2.5-coder:32b | 0.902 | 1.000 | 0.940 | 0.730 |
| gemma3:27b | 0.878 | 0.318 | 0.955 | 1.000 |
| qwen2.5:7b | 0.829 | 0.0024 | 0.910 | 0.038 |

*`gemini-2.5-pro` = `single_strong`; **bold** = TOP on each benchmark. Exact McNemar vs. the frontier; Bonferroni $n_{\text{compare}}$=5. Only qwen2.5:7b is significantly *worse* than the frontier; no model beats it.

**Table 9**: **Public-benchmark arm: saturation reverses finding #2.** Single-sample accuracy on HumanEval ($n$=164) and GSM8K ($n$=200) for six models, with an exact McNemar test of each model against the frontier (`gemini-2.5-pro`, Bonferroni $n_{\text{compare}}$=5). The frontier generalist is TOP on *both* benchmarks; specialists tie or are significantly worse. On saturated tasks the test-time-selection effect of §9.2 vanishes — standard public benchmarks are too easy to separate these models.

**Two compounding mechanisms.** (i) *Saturation:* HumanEval and GSM8K are near-ceiling for this class of model, so they cannot separate them, and any test-time-selection effect washes out. The re-verification above shows the hard-coding gap is real (+0.. + 2/33), so the disappearance is genuinely *hard-task-specific*, not a verifier bug: pick a saturated benchmark and you manufacture

"no effect"; pick a hard one and you manufacture "big effect." (ii) *Underpowered nulls:* our verifiable backbone has only $n$=68 tasks, and an exact McNemar at that size needs a 72–100% discordant split before it can reject at $\alpha$=0.05 — so a bare "no significant difference" there carries almost no information either way (the judge-null itself is a failure to reject, point estimate $d = +0.103$, CI $[-0.020, +0.226]$, not equivalence). **What would have caught it:** use hard, non-saturated tasks *and* calibrate on a public benchmark to expose saturation; report power or an equivalence interval rather than a bare null (§10).

## 10 A Trustworthy-Evaluation Protocol

The three findings were not produced by carelessness; each was internally consistent and survived ordinary review. What kills them is binding the analysis to something the analyst cannot rationalize. We distill the protocol as a checklist, each item tagged with the manufactured finding it would have caught.

1. **Verify the verifier (catches #1).** Before trusting any model score, run the benchmark's own reference solutions, and a few hand-written known-good/known-bad cases, through your grading harness; confirm it passes the good and fails the bad. A grader bug is indistinguishable from a model finding until you do this.

2. **Bind to a correct external oracle (the robust result).** The only selector that beat `single_strong` was the executor, and only because it recomputes the gold answer. Prefer an external, ground-truth-bearing check (execution, a programmatic validator) over any model-on-model verdict, which inherits the generator's blind spots.

3. **Use hard, non-saturated tasks — and calibrate on a public benchmark (catches #3).** Saturated benchmarks hide selection effects; arbitrarily hard in-house tasks can invent them. Report both, so the reader can see whether an effect is task-difficulty-specific.

4. **Stratify by domain (catches #2, and exposes it).** Aggregate nulls hide Simpson reversals; stratification reveals the heterogeneity — but then you must pay for the multiple looks (next item) and state the claim the strata actually support.

5. **Report power / equivalence, not bare nulls (catches #3's null).** "No significant difference" at $n$=68 is nearly vacuous when the design needs a 72–100% discordant split to detect anything. Report the minimum detectable effect or a TOST equivalence interval; apply multiplicity correction to every stratified comparison.

6. **Pre-register (catches all three).** Fix the verifier, the task set, the comparison, and the analysis before looking; label everything else exploratory. Each of our three findings exploited a degree of freedom that pre-registration removes.

## 11 Discussion

**Verification asymmetry applies reflexively to the researcher.** The mechanism we invoke for why a model is a poor judge of another model — between systems of the same kind, checking and solving are the same operation over the same evidence, so the checker inherits the generator's blind spots [7] — is the same reason a researcher is a poor judge of their own selection result. Confirming "did orchestration really win?" is, by default, the same analysis that produced the win, run again,

so it imports the same artifacts: a grader bug (#1), a favorable stratification (#2), a convenient benchmark or an underpowered null (#3). Each of our three findings was internally consistent and survived ordinary scrutiny; what broke them was binding the analysis to something external the analyst could not rationalize — the benchmark's own reference solutions, a multiplicity correction, a hard-vs-saturated contrast, a power calculation. The single robust positive — the executor — wins precisely because it is that external thing: it recomputes the gold answer rather than rendering a verdict, and its 1.000 is true by construction, not a contested margin.

**Why no model-on-model selector helped (one-line mechanism).** Between models of the same kind verification is not asymmetric: the checker has no oracle access and its verdict is correlated with the generator's errors, so it adds variance rather than information. This is the parsimonious explanation for the E-B judge-null and for why an *external* verifier — a different kind of operation — reaches the ceiling. We offer it as a mechanism, not a law.

**Capability is not the cure either.** A common objection to any judge-null is that the judges were not strong enough. E-C answers it: raw strength is the wrong axis. The strongest rung (`gemini-2.5-pro`) is numerically below several mid-tier models on the hard suite, yet TOP on both saturated public benchmarks; the self-identity confound in the strongest "judge" rung (the *identical model* to `single_strong`, $(6, 5)$, $p$=1.0) is removed by the distinct qwen2.5-coder:32b judge, which still fails to reach significance. Scaling capability did not turn a model into a reliable judge of another model.

**An illustrative anecdote (untested): the strongest model can hallucinate arithmetic.** As a *worse*-than-baseline selector, `gemini-2.5-pro` is numerically below several mid-tier models (not significantly so on $n$=68). We offer one untested mechanism (two samples, not a significance test): on tasks turning on *exact* arithmetic (e.g. a seven-digit multiplication), it returned two *different* wrong 14-digit products, each narrating a "calculator" it never invoked. Fluency can raise the *confidence* of a wrong answer without raising its correctness. This is a suggestive anecdote, not a finding, and we do not build any claim on it.

**Coverage × Selectability localizes the problem (here illustratively).** Measuring Coverage first tells a practitioner whether to spend compute on generation or selection; here Coverage is saturated, so the problem is selection. We repeat the scope caveat: on this backbone Coverage is a constant 1.000, so Selectability = acc identically and the decomposition is *illustrated*, not tested; whether the factor that binds is Coverage or Selectability is a property of the suite, not a prediction.

**Practical guidance (scoped to this regime).** (1) Before believing any test-time-selection win, run the protocol of §10 — verify the verifier, bind to an external oracle, use hard *and* public tasks, stratify, report power, pre-register. (2) Do *not* spend the budget on a better model-on-model judge — of any strength or identity we tested, it did not reliably beat a single strong model; budget for it as a tie at best. (3) Where the answer is externally checkable, bind to the checker as a ceiling, noting the apparent efficiency of routing only the contested tasks is an in-sample property (§8.2). (4) Treat an in-house "orchestration beats the model" headline as a hypothesis to be falsified, not a result — especially when it is yours.

## 12 Limitations

We state the boundaries plainly. **(i) Single, unseeded run.** The backbone is one cached pool; generation is unseeded, so results reproduce over the committed pool, not by regeneration. The reported Wilson intervals and McNemar tests reflect *task-sampling* uncertainty only (conditional on the single cached pool) and do *not* bound run-to-run variability; a re-draw could flip the small discordant splits that drive the within-domain coding tests ($+13/-3$, $+12/-3$, $+11/-2$). **(ii) Small $n$, small per-domain counts.** The suite is 68 tasks (26 arithmetic, 33 coding, 9 constrained); constrained generation in particular is only 9 tasks, too few to test on its own. The significant effects are within coding ($n$=33). **(iii) Multiplicity on the within-domain tests.** The three coding specialist-vs-baseline tests are correlated; Bonferroni $\times 3$ lifts the nominal 0.021/0.035/0.023 to $\sim$0.06–0.10. We rest the claim on the 3/3 directional conjunction, not a single starred result. **(iv) The combined router is underpowered.** Out-of-sample the domain router (0.912) beats the best fixed model (0.824) only directionally ($+4/-1$, $p$=0.38, $n_{\text{test}}$=34). The per-domain effects are significant but the end-to-end router needs *more tasks per domain* to power (future work). **(v) Underpowered judge-null; not equivalence.** The judge-null rests on few discordant pairs; several judges are numerically *above* the baseline. We claim no significant judge gain and that judging does not recover the domain prior, not that judging hurts and not equivalence (point estimate $d = +0.103$, CI $[-0.020, +0.226]$; no TOST against a pre-specified margin). **(vi) Bounded model set.** Only eight open+frontier models, topping out at a non-reasoning Gemini tier; dedicated reasoning/verification models (OpenAI o-series, DeepSeek-R1, Claude) were not evaluated. **(vii) Verifiable suite only.** The tasks are arithmetic, coding, and constrained generation, all chosen for clean external verification; the executor reference is exact *by construction* and would be messier where verification is partial. The `none` (no-verifier) regime and the gating/optimization regimes (§2) are future work. **(viii) Illustrative decomposition; in-sample routing.** With Coverage a constant 1.000 the Coverage $\times$ Selectability split is illustrative, not independently tested; the disagreement-gated executor operating point (§8.2) is in-sample. **(ix) Our own suite, and a reflexive caveat.** The manufactured findings are demonstrated on our own tasks, pipeline, and one frontier tier; we make no claim that these are the *only* mechanisms, and the cautionary thesis applies to this paper too — the protocol of §10 is the standard we hold ourselves to, not a result we have independently replicated. With these bounds stated, the robust conclusion is conservative: on this suite, no model-on-model selector significantly beat a single strong model, and only an external executor reached the ceiling, by construction.

## 13 Conclusion

We treated the test-time harness — judge, voter, router, self-improvement loop — as an engineering artifact and distilled a compact set of design principles for making one trustworthy (§4): bind to an executable oracle; select by execution, not self-certification; hold out the labels the selector uses; gate rather than rank; calibrate difficulty and report power; stratify and respect multiplicity; run seeds; require diversity before a committee. Each principle was earned, not asserted. We studied test-time selection among cached candidates on 68 verifiable tasks (26 arithmetic, 33 coding, 9 constrained generation; open models up to one frontier API tier; one unseeded run) plus a public-benchmark arm (HumanEval $n$=164, GSM8K $n$=200), and showed that *violating* a principle manufactures a false "test-time alpha" finding three ways in our own data — a verification-grader bug (gemini-2.5-pro 0/3 $\rightarrow$ 12/12; violates P1), a Simpson/aggregation artifact (coding specialists $p$=0.021/0.035/0.023 that fail Bonferroni and do not replicate; P6), and benchmark saturation with

an underpowered null (the frontier is TOP on both public benchmarks; an exact McNemar at $n$=68 needs a 72–100% discordant split to detect anything; P5). The same verification asymmetry that makes a model a poor judge of a model makes a researcher a poor judge of their own selection result — which is why the principles double as an evaluation protocol (§10). The one robust result survives all of it, and only because it is external and by-construction: *model-on-model selection does not reliably beat a single strong model; only external execution does.* And the same asymmetry that audits a false result can *construct* a good one: an execution-gated best-of-$N$ harness on local models builds a competitive pipeline where model self-certification yields fiction or a null (§A). Engineer the harness, not the model.

# A Companion experiment: self-improvement (optimize regime) on local models

**This is a concurrent companion experiment by the present author, in the *optimize* regime this paper does not test; like the observations of §2 it is *motivation*, not evidence for any claim here, and its numbers are not derived from the released `data/ea/` backbone.** We record it because a self-improvement loop reproduces the same asymmetry the selection regime exhibits: a model asked to certify or improve a model's own work adds little reliably, while *binding to an external executor and grader* is what separates real progress from apparent progress.

**Setup.** We port SIA [6], a self-improving agent framework — a meta-agent writes a task-specific target agent; the target attempts the task; a feedback agent revises it across *generations* — to local open-weight models served through Ollama's OpenAI-compatible endpoint (`qwen3-coder:30b`, `gemma4`). The task is Kaggle *spaceship-titanic* (tabular binary classification; random $\approx 0.50$, competent pipelines $\approx 0.79$–$0.82$). Each run executes three self-improvement generations; we score each generation's `submission.csv` against held-out labels and take the best accuracy per run. A run is *competitive* if some generation exceeds 0.5.

**Two observations matching this paper's pattern.** *(i) An unverified self-report is a verification artifact.* Under SIA's default design the meta-agent writes a target that is itself a nested LLM-agent. On several runs this nested agent *reported* success in prose — e.g. "Achieved validation accuracy 0.7942; created submission.csv" — while executing nothing and writing no file; only external execution and grading against gold revealed the claimed score was fiction. This is the verification-artifact mechanism of Manufactured Finding #1 (§9): the grader, not the self-report, tells the truth. *(ii) Model-on-model revision is unreliable; external binding is what moves outcomes.* The feedback agent's generation-over-generation revision rarely improved a working pipeline and sometimes regressed it (e.g. $0.813 \rightarrow 0.792$). What changed yields was a structural intervention that *binds the target to an executor*: directing the meta-agent to emit a plain, directly-executed script rather than a nested agent (Fig. 4) lifted the competitive-run fraction from 1/5 to 2/3 for `qwen3-coder` and from 0/3 to 1/3 for `gemma4`. The residual failures then become mundane code bugs (syntax errors, wrong label encoding), not the structural API-incompatibility that dominated the default.

*(iii) The reproducible win is the execution **gate**, not the committee.* Pushing the binding one step further — sampling $N$ candidate targets and keeping only those that actually run and score on a self-held-out validation split whose labels the harness withholds — converts an unreliable generator into a dependable one: the gate never ships a non-running or regressed solution (a

broken candidate scores `None` and is excluded), and at $N$=8 this delivered a competitive pipeline on ∼0.80–0.82 in every run. We then tested whether *ensembling* the gate-passers — a majority vote of the executable-selected committee — buys accuracy over simply taking the single validation-best survivor. Across four seeds it did *not*: ensemble vs. single-best delivered 0.807 vs. 0.806 on average (per-seed $\Delta = +0.005, 0, 0, 0$), a null. The reason is instructive and consistent with this paper's framing: best-of-8 typically left only ∼2 survivors, and the survivors were near-identical `sklearn` pipelines whose errors are correlated, so a committee adds almost nothing. The lift came entirely from the *selection* step — an external executor deciding which candidate runs and scores — not from any model-level aggregation, exactly the asymmetry §3 predicts. (We flag this as a self-caught instance of the paper's own thesis: a single favorable draw showed the committee at 0.821 vs. 0.816; only running additional seeds revealed it as variance, not an effect.)

*(iv) The selector must be an executor, not a judge — which reconciles the "verification hurts" paradox.* Holding the candidate pool fixed, we vary only *how* the best-of-$N$ winner is chosen and score the chosen candidate on the private gold (Table 10). Taking the first draw (no selection) averages 0.727; a *model-judge* that reads the candidate scripts and picks one averages 0.786, only modestly above a random pick (0.760) and worse than the gate on two of five runs (0.768 vs. 0.808; 0.753 vs. 0.805); the *executable gate* (argmax validation score) averages 0.804, essentially the hindsight oracle (0.805). So execution-based selection captures nearly all the available headroom while model-judgment selection captures about half of it. This is the harness literature's "verification loops degrade performance" paradox resolved at the level of mechanism: a model verifying a model adds noise and barely beats chance, whereas an executable verifier is at the ceiling — the same execution-vs-judgment distinction, now measured inside the loop (small-$n$, illustrative).

| Selection mode | none | random | model-judge | exec. gate | oracle |
|---|---|---|---|---|---|
| Private-gold accuracy | 0.727 | 0.760 | 0.786 | **0.804** | 0.805 |

**Table 10**: *How you select among best-of-N candidates* (mean over the five runs with ≥ 2 candidate pipelines), scored on the private test gold. The executable gate (0.804) reaches the hindsight oracle (0.805); a model-judge (0.786) lands nearer random (0.760). Small-$n$, single-task; the ordering is the point.

*(v) Vary a different knob and the fragility returns — so we resist the finding.* Holding selection fixed and varying only the agent's turn/context budget (`MAX_TURNS`$\in \{25, 50, 100\}$, three seeds each) produces a striking apparent effect: the low budget delivers a competitive pipeline in 3/3 runs (mean 0.796) while the high budget delivers in 0/3. Read by outcome alone this is a dramatic "a smaller context budget is better" harness law. It is not one. The high-budget failures are ordinary code-quality crashes in the generated scripts (an `sklearn` label-type error; a script that reads `train_inner.csv` from the wrong directory), and at three seeds the design cannot separate a plausible "more turns over-engineer the script" mechanism from variance. We therefore claim no budget law — only that a *second* harness knob, naively analyzed, again manufactures a large unreplicated effect (violating P5 and P7). The contrast with (iv) is the whole point: across the knobs we varied, the only intervention that produced a *robust* gain was binding selection to an executor; every knob judged by outcome alone produced an artifact — including, reflexively, our own.

**Caveat.** These remain small-$n$, single-task companion observations, not measured results of this paper. The direction is the point, and it is consistent end-to-end: a model certifying, revising, or aggregating a model was unreliable (and at times fictional, or merely a null), whereas what reliably moved outcomes — from producing a valid pipeline at all, to keeping the delivery in the

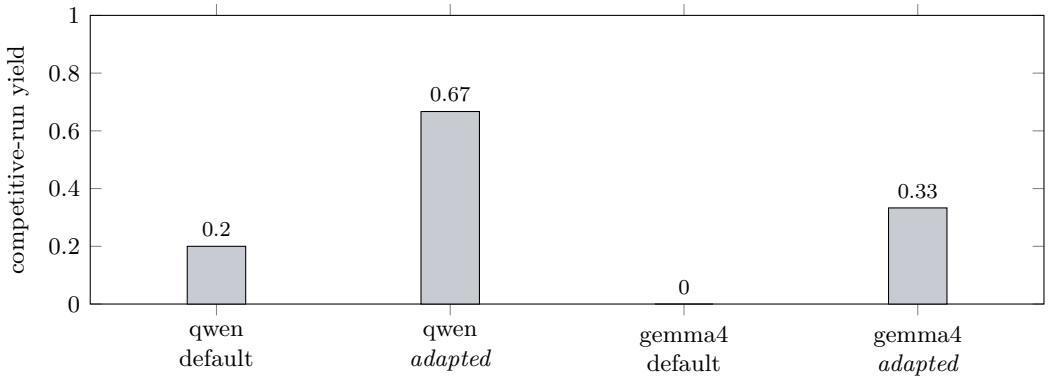


**Figure 4**: *Companion observation (motivation, not a measured result of this paper).* Fraction of SIA self-improvement runs that produced a competitive spaceship-titanic pipeline ($> 0.5$ accuracy on held-out labels), by model and by target design. "Adapted" replaces SIA's default nested-LLM-agent target — whose self-reports are unverified and which fails against the local endpoint's stricter message validation — with a plain, directly-executed script that binds the model's work to an external interpreter and grader. Counts: qwen-default 1/5, qwen-adapted 2/3, gemma-default 0/3, gemma-adapted 1/3. Yields are small-$n$ and high-variance; the point is directional.

competitive band — was binding *selection* to an external executor and grader. That is the same asymmetry this paper measures in the selection regime, now seen to *construct* as well as *audit*: the executable oracle is how one builds a dependable result, while model-level aggregation on top of it added nothing measurable here.

## Declaration of generative AI use

During the preparation of this work the author used *Claude Code (Anthropic)* for implementing the shared backbone and analysis, generating and debugging analysis and table-rendering code, and drafting manuscript boilerplate. All model-generated candidates in the pool are produced by the local and API models named in §6; all gold answers are computed symbolically in Python, never by a model. After using these tools the author reviewed and edited all content and takes full responsibility for the publication. No generative-AI system was used to fabricate data or results, and no AI tool is listed as an author.

## Data and code availability

All experiment code, the cached candidate pool (`data/ea/ea_pool_full.jsonl`, 4,896 records), the content-hashed call caches, the generated tables (`paper/tables/{domain,eb,ec,ee,ef,bench_public}.tex`), the domain analysis (`data/ea/domain_analysis.txt`, reproducible via `scripts/run_domain.py` and `scripts/run_tier_ab.py`), the public-benchmark results (`data/ea/bench_{humaneval,gsm8k}.txt` via `scripts/run_benchmark.py`), and the analysis scripts are available at github.com/jim-schwoebel/verification-asymmetry-paper. Analysis is deterministic over the committed pool; generation is unseeded, so the pool — not a seed — is the reproducibility artifact.

## Author contributions

J.S. conceived the study, implemented the shared backbone and analysis, ran the experiments, and wrote the manuscript.

## Competing interests

The author declares no competing interests.

## Funding

This work received no specific external funding.

## References


[1] Anonymous. SkillRevise: Improving LLM-authored agent skills via trace-conditioned skill revision. *arXiv preprint arXiv:2606.01139*, 2026.

[2] Lingjiao Chen, Matei Zaharia, and James Zou. FrugalGPT: How to use large language models while reducing cost and improving performance. In *Transactions on Machine Learning Research (TMLR)*, 2024. arXiv:2305.05176.

[3] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.

[4] Yilun Du, Shuang Li, Antonio Torralba, Joshua B. Tenenbaum, and Igor Mordatch. Improving factuality and reasoning in language models through multiagent debate. *arXiv preprint arXiv:2305.14325*, 2023.

[5] Luyu Gao, Aman Madaan, Shuyan Zhou, Uri Alon, Pengfei Liu, Yiming Yang, Jamie Callan, and Graham Neubig. PAL: Program-aided language models. In *International Conference on Machine Learning (ICML)*, 2023. arXiv:2211.10435.

[6] Sharath Hebbar et al. SIA: A self-improving agent framework for autonomous task solving. *arXiv preprint arXiv:2605.27276*, 2026.

[7] Jie Huang, Xinyun Chen, Swaroop Mishra, Huaixiu Steven Zheng, Adams Wei Yu, Xinying Song, and Denny Zhou. Large language models cannot self-correct reasoning yet. 2024. arXiv:2310.01798.

[8] Subbarao Kambhampati. Can large language models reason and plan? *Annals of the New York Academy of Sciences*, 1534(1):15–18, 2024.

[9] Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let's verify step by step. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2305.20050.

[10] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. Self-refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. arXiv:2303.17651.

[11] Mahdi Naser Moghadasi et al. What twelve LLM agent benchmark papers disclose about themselves: A pilot audit and an open scoring schema. *arXiv preprint arXiv:2605.21404*, 2026.

[12] Xuying Ning et al. Code as agent harness: Toward executable, verifiable, and stateful agent systems. *arXiv preprint arXiv:2605.18747*, 2026.

[13] Arjun Panickssery, Samuel R. Bowman, and Shi Feng. LLM evaluators recognize and favor their own generations. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. arXiv:2404.13076.

[14] Jim Schwoebel. Agents as a judge: Hard-voting panels, the condorcet boundary, and the generator bottleneck. Technical report, Quome, 2026. Companion study (SELECT regime, Condorcet boundary).

[15] Jim Schwoebel. The gate, not the judge: Why judge composition does not move correctness and an asymmetric-loss ground-truth gate does. Technical report, Quome, 2026. Companion study (GATE regime).

[16] Jim Schwoebel. Objective drift in loop-as-a-judge: When iterative LLM optimization optimizes the wrong target. Technical report, Quome, 2026. Companion study (OPTIMIZE regime).

[17] Jim Schwoebel. Where test-time alpha comes from: Coverage × selectability and an executable-oracle dichotomy for LLM committees. Technical report, Quome, 2026. Companion study (SELECT regime).

[18] Kaya Stechly, Karthik Valmeekam, and Subbarao Kambhampati. On the self-verification limitations of large language models on reasoning and planning tasks. In *International Conference on Learning Representations (ICLR)*, 2025. arXiv:2402.08115.

[19] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. In *International Conference on Learning Representations (ICLR)*, 2023. arXiv:2203.11171.

[20] Zhongwei Xie et al. A survey on AI agent harness. *arXiv preprint*, 2026.

[21] Yilun Yao et al. Harness-Bench: Measuring harness effects across models in realistic agent workflows. *arXiv preprint arXiv:2605.27922*, 2026.

[22] Murong Yue, Jie Zhao, Min Zhang, Liang Du, and Ziyu Yao. Large language model cascades with mixture of thought representations for cost-efficient reasoning. *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2310.03094.

[23] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging LLM-as-a-judge with MT-bench and chatbot arena. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2023. arXiv:2306.05685.